



UNIVERSITY OF BUEA
FACULTY OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF COMPUTER ENGINEERING

CEF440: INTERNET PROGRAMMING AND MOBILE PROGRAMMING

**DESIGN AND IMPLEMENTATION OF A MOBILE APP FOR COLLECTION OF USER
EXPERIENCE DATA FROM MOBILE NETWORK SUBSCRIBERS.**

TASK 6: DATABASE DESIGN AND IMPLEMENTATION

GROUP 2

COURSE INSTRUCTOR: Dr. Valery Nkemeni

S/N	NAME	MATRICULE
1.	EFUETNGONG GENESIS TAZISONG	FE23A041
2.	MATUTE MOLUA FRITZ	FE23A087
3.	NWANTOLY STEPHILL FAVOUR BENYELLA	FE23A127
4.	NYANYOH DIVINE-FORTUNE BANFILA	FE23A128
5.	TENKEN MANPLA ULRICH	FE23A170

Contents

1. INTRODUCTION	2
2. DATA ELEMENTS.....	2
3. CONCEPTUAL DESIGN	4
3.1 Entities and Their Roles	4
Entity 2: NetworkMetric.....	4
Entity 3: Feedback.....	4
Entity 4: AppSettings (Local Only)	4
3.2 Relationships	4
3.3 Key Design Decisions	5
4. Entity Relationship (ER) Diagram.....	6
5. DATABASE IMPLEMENTATION	7
5.1 Local Database Android Room (SQLite)	7
5.1.1 Entity Classes.....	7
5.1.2 Data Access Objects (DAOs).....	7
5.1.3 Database Class	8
5.1.4 Room Database Verification	8
5.2 Remote Database MySQL on Railway	10
5.2.1 MySQL Schema	10
5.2.2 MySQL Tables Verification	11
6. BACKEND IMPLEMENTATION.....	12
6.1 Technology Stack.....	12
6.2 Project Structure	12
6.3 API Endpoints	12
6.4 Sequelize Models.....	12
6.5 Backend Deployment Verification.....	13
7. CONNECTING THE DATABASE TO THE BACKEND.....	15
7.1 Architecture Overview	15
7.2 Retrofit HTTP Client (Android).....	15
7.3 Sync Worker	16
7.4 Sequelize Database Connection	16
7.5 End-to-End Sync Verification	17
8. SUMMARY AND CONCLUSION	19
8.1 Deliverables Summary	19
8.2 Technical Achievements	19
8.3 Limitations and Future Work	19

1. INTRODUCTION

This report documents the complete database design and implementation for the QoETRACK mobile application developed as part of the CEF440 group project. QoETRACK is an Android application designed to collect real-time Quality of Experience (QoE) data from mobile network subscribers in Cameroon, combining passive network metric collection with active user feedback.

The database architecture is designed as a two-tier system: a local Room/SQLite database embedded within the Android application for offline-first operation, and a remote MySQL database hosted on Railway cloud infrastructure accessible through a RESTful Express.js backend API. This document covers all required sub-tasks: Data Elements, Conceptual Design, ER Diagram, Database Implementation, Backend Implementation, and Connecting the Database to the Backend.

2. DATA ELEMENTS

The following table identifies every data element the QoETRACK application needs to store, grouped by entity. Connectivity Manager, Fused Location Provider Client) and actively (from the user via the Feedback screen). Entities marked with an asterisk (*) exist only in the local Room database and are not synchronised to the backend.

Entity	Field Name	Data Type	Constraint	Description
User	user_id	INT	PK, AUTO_INC	Unique identifier for each subscriber
User	device_id	VARCHAR(100)	UNIQUE, NOT NULL	Unique device identifier (ANDROID_ID)
User	created_at	DATETIME	DEFAULT NOW()	Timestamp of first app registration
NetworkMetric	metric_id	INT	PK, AUTO_INC	Unique reading identifier
NetworkMetric	user_id	INT	FK → users	References the subscriber
NetworkMetric	network_type	VARCHAR(20)	NOT NULL	Connection type (4G LTE, WiFi, 5G, 3G)
NetworkMetric	signal_strength	INT		Signal strength in dBm (e.g. -87)
NetworkMetric	latency	INT		Round-trip latency in milliseconds
NetworkMetric	jitter	INT		Latency variation in milliseconds
NetworkMetric	packet_loss	FLOAT		Estimated packet loss percentage
NetworkMetric	bandwidth	FLOAT		Downstream bandwidth in Kbps
NetworkMetric	latitude	DOUBLE		GPS latitude coordinate
NetworkMetric	longitude	DOUBLE		GPS longitude coordinate
NetworkMetric	recorded_at	DATETIME	DEFAULT NOW()	Timestamp of data collection
NetworkMetric	synced	BOOLEAN	DEFAULT FALSE	Whether row has been pushed to backend

Feedback	feedback_id	INT	PK, AUTO_INC	Unique feedback entry identifier
Feedback	user_id	INT	FK → users	References the subscriber
Feedback	overall_satisfaction	INT	1–5	Star rating for overall network quality
Feedback	speed_rating	INT	0–100	Slider value for perceived speed
Feedback	usability_rating	INT	0–100	Slider value for app/network usability
Feedback	comments	TEXT	NULLABLE	Optional free-text user comments
Feedback	submitted_at	DATETIME	DEFAULT NOW()	Timestamp of feedback submission
Feedback	synced	BOOLEAN	DEFAULT FALSE	Whether row has been pushed to backend
AppSettings*	id	INT	PK = 1	Single-row settings record
AppSettings*	dark_mode_enabled	BOOLEAN	DEFAULT FALSE	User's dark/light theme preference
AppSettings*	bg_collection_enabled	BOOLEAN	DEFAULT TRUE	Whether background data collection is on
AppSettings*	prompt_frequency_hrs	INT	DEFAULT 6	Hours between feedback prompt notifications

* *AppSettings* is a local-only entity stored in Room/SQLite on-device. It holds user preferences and does not contain personal or network data requiring server-side storage.

3. CONCEPTUAL DESIGN

The conceptual design defines the logical structure of the database independently of any specific database technology. It identifies entities, their attributes, primary and foreign keys, and the relationships between them.

3.1 Entities and Their Roles

Four entities are identified in the QoETRACK system:

Entity 1: User

Represents a registered mobile subscriber. Each device registers once upon first launch. The User entity is the central entity all collected data (metrics and feedback) is associated with a user.

- Primary Key: user_id (auto-incremented integer)
- Key attribute: device_id (unique device identifier ensuring one registration per device)

Entity 2: NetworkMetric

Represents a single passive data collection event recorded by the background WorkManager worker. A new row is inserted every 15 minutes while the app is running. Contains both objective network measurements (signal strength, latency, jitter, bandwidth, packet loss) and spatial context (GPS coordinates).

- Primary Key: metric_id
- Foreign Key: user_id → references User
- Contains synced flag to track upload status to the backend

Entity 3: Feedback

Represents a subjective quality rating submitted by the user through the Feedback screen. Captures star ratings, slider-based dimension scores (speed, usability), and optional free-text comments.

- Primary Key: feedback_id
- Foreign Key: user_id → references User
- Contains synced flag to track upload status to the backend

Entity 4: AppSettings (Local Only)

A single-row entity (id always = 1) storing user preferences on-device. Not synchronised to the backend as it contains only UI preferences with no analytical value to network operators.

- Primary Key: id (always 1 singleton pattern)
- Stores: dark mode toggle, background collection toggle, prompt frequency

3.2 Relationships

Relationship	From	To	Description
generates	User (1)	NetworkMetric (N)	One subscriber generates many metric readings over time
submits	User (1)	Feedback (N)	One subscriber submits many feedback entries over time
has	User (1)	AppSettings (1)	One subscriber has one local settings record (Room only)

3.3 Key Design Decisions

- Offline-first architecture: All data is written to local Room/SQLite first, then asynchronously pushed to the remote MySQL database via WorkManager ensuring data is never lost due to network unavailability.
- Synced flag pattern: Both NetworkMetric and Feedback entities carry a boolean synced column. The SyncWorker queries for unsynced rows, posts them to the backend, and marks them synced=true only on a confirmed HTTP 201 response.
- Single-user model: The current implementation uses a single user registration per device (device_id as unique key), keeping the schema simple for the research context.
- Snake_case convention: All database column names use snake_case (e.g. signal_strength, recorded_at) consistent between the Android Room entities, Sequelize models, and MySQL schema.

4. Entity Relationship (ER) Diagram

The Entity Relationship Diagram below illustrates the four entities, their attributes (with primary and foreign keys indicated), and the relationships between them. Crow's foot notation is used: a single vertical line represents 'one' and a crow's foot represents 'many'.



Figure 1: QoETRACK ER Diagram generated from qoetrack_er_diagram.html

The diagram clearly shows:

- Users as the central/parent entity with **user_id** as primary key
- NetworkMetric and Feedback as child entities with **user_id** as foreign key (ON DELETE CASCADE)
- 1:N (one-to-many) relationships from User to both NetworkMetric and Feedback
- AppSettings as an isolated local-only entity with no foreign key relationships

5. DATABASE IMPLEMENTATION

The QoETRACK database is implemented at two levels: a local Room/SQLite database embedded within the Android application, and a remote MySQL database hosted on Railway cloud infrastructure.

5.1 Local Database Android Room (SQLite)

Room is Google's abstraction layer over SQLite, providing compile-time SQL query validation, LiveData/Flow integration for reactive UI updates, and type safety. The local database is named `qoetrack_local_db`.

5.1.1 Entity Classes

Each database table is represented by a Kotlin data class annotated with `@Entity`:

```
// NetworkMetricEntity.kt
@Entity(tableName = "network_metrics")
data class NetworkMetricEntity(
    @PrimaryKey(autoGenerate = true) val metricId: Long = 0,
    val networkType: String,
    val signalStrength: Int,
    val latency: Int,
    val jitter: Int,
    val packetLoss: Float,
    val bandwidth: Float,
    val latitude: Double,
    val longitude: Double,
    val recordedAt: Long,
    val synced: Boolean = false
)
```

5.1.2 Data Access Objects (DAOs)

DAOs define all SQL operations used by the application:

```
@Dao
interface NetworkMetricDao {
    @Insert
    suspend fun insert(metric: NetworkMetricEntity)

    @Query("SELECT * FROM network_metrics WHERE synced = 0")
    suspend fun getUnsynced(): List<NetworkMetricEntity>

    @Query("UPDATE network_metrics SET synced = 1 WHERE metricId = :id")
    suspend fun markSynced(id: Long)

    @Query("SELECT * FROM network_metrics ORDER BY recordedAt DESC LIMIT 1")
    fun observeLatest(): Flow<NetworkMetricEntity?>
}
```



```

    @Query("SELECT AVG(signalStrength) FROM network_metrics WHERE recordedAt
    >= :since")
    suspend fun getAverageSignal(since: Long): Double?
}

```

5.1.3 Database Class

The abstract RoomDatabase class registers all entities and provides DAO access:

```

@Database(
    entities = [NetworkMetricEntity::class, FeedbackEntity::class, AppSettingsEntity::class],
    version = 2,
    exportSchema = false
)
abstract class QoETrackDatabase : RoomDatabase() {
    abstract fun networkMetricDao(): NetworkMetricDao
    abstract fun feedbackDao(): FeedbackDao
    abstract fun appSettingsDao(): AppSettingsDao

    companion object {
        @Volatile private var INSTANCE: QoETrackDatabase? = null
        fun getInstance(context: Context): QoETrackDatabase { ... }
    }
}

```

5.1.4 Room Database Verification

The following screenshot from Android Studio's App Inspection → Database Inspector confirms the local database is operational with real data collected from the device in Buea, Cameroon:

The screenshot shows the Android Studio Database Inspector with the `network_metrics` table selected. The table contains 50 rows of data. The columns are: `id`, `me...`, `network...`, `signalS...`, `latency`, `jitter`, `packet...`, `bandwi...`, `latitude`, `longitu...`, `record...`, and `synced`.

id	me...	network...	signalS...	latency	jitter	packet...	bandwi...	latitude	longitu...	record...	synced
35	35	WIFI	-92	29	13	0.0	30268.0	4.1556815	9.2862023	1781449270	1
36	36	WIFI	-92	32	5	0.0	30268.0	4.1556815	9.2862023	1781449270	1
37	37	WIFI	-92	46	18	0.0	30268.0	4.1556815	9.2862023	1781449270	1
38	38	WIFI	-92	38	8	0.0	30268.0	4.1556815	9.2862023	1781449270	1
39	39	WIFI	-92	37	85	0.0	30268.0	4.1556815	9.2862023	1781449271	1
40	40	WIFI	-92	37	9	0.0	30268.0	4.1556815	9.2862023	1781449271	1
41	41	WIFI	-92	44	1	0.0	30268.0	4.1556815	9.2862023	1781449271	1
42	42	WIFI	-92	34	6	0.0	30268.0	4.1556815	9.2862023	1781449271	1
43	43	WIFI	-92	179	103	0.0	30268.0	4.1556815	9.2862023	1781449272	1
44	44	4G LTE	-92	42	45	0.0	14.0	4.1556815	9.2862023	1781449717	1
45	45	WIFI	-95	530	181	0.0	30268.0	4.1556731	9.2862056	1781449746	1
46	46	WIFI	-87	50	11	0.0	30268.0	4.1556731	9.2862056	1781449847	1
47	47	WIFI	-87	115	61	0.0	30268.0	0.0	0.0	1781450179	1
48	48	WIFI	-90	45	4	0.0	30268.0	4.1556731	9.2862056	1781450334	1
49	49	WIFI	-90	49	8	0.0	30268.0	4.1556731	9.2862056	1781450335	1
50	50	WIFI	-98	33	1	0.0	30268.0	4.1556731	9.2862056	1781450760	1

Figure 2: Android Studio Database Inspector `network_metrics` table showing 50+ real rows with GPS coordinates (lat ≈ 4.1556 , lon ≈ 9.2862), signal strength, and latency values

The screenshot shows the Android Studio Database Inspector with the `feedback` table selected. The table contains 10 rows of data. The columns are: `id`, `feedbackId`, `overallSatisfac...`, `speedRating`, `usabilityRating`, `comments`, `submittedAt`, and `synced`.

id	feedbackId	overallSatisfac...	speedRating	usabilityRating	comments	submittedAt	synced
3	3	60	48	100	bad	1781448376811	1
4	4	60	48	100	bad bad	1781448794342	1
5	5	60	48	100	the network is very	1781449293372	1
6	6	40	48	100	NULL	1781449703973	1
7	7	60	48	100	not the best	1781450449384	1
8	8	60	48	100	wifi	1781450817633	1
9	9	40	48	17	NULL	1781451279962	1
10	10	60	48	100	very poor	1781452503740	1
11	11	60	48	100	geno	1781454076963	1
12	12	60	48	100	hello	1781458237358	1
13	13	60	48	100	hello	1781464690532	1
14	14	80	0	0	he'll	1781464698088	1
15	15	40	48	100	the network is poor	1781515177215	1
16	16	60	48	100	I am not connected	1781515557201	1
17	17	60	48	100	Using Starlink	1781515957775	1
18	18	60	48	100	my network is much	1781546628569	1

Figure 3: Android Studio Database Inspector `feedback` table showing 10 user-submitted entries with ratings and comments

5.2 Remote Database MySQL on Railway

The remote database uses MySQL 8.0 hosted on Railway cloud infrastructure. It stores the same data as the local Room database but serves as the centralised repository accessible to network operators via the web dashboard.

5.2.1 MySQL Schema

The following SQL statements define the remote database schema:

```
CREATE DATABASE qoetrack_db;
USE qoetrack_db;

CREATE TABLE users (
  user_id INT AUTO_INCREMENT PRIMARY KEY,
  device_id VARCHAR(100) UNIQUE NOT NULL,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE network_metrics (
  metric_id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  network_type VARCHAR(20),
  signal_strength INT,
  latency INT,
  jitter INT,
  packet_loss FLOAT,
  bandwidth FLOAT,
  latitude DOUBLE,
  longitude DOUBLE,
  recorded_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  synced BOOLEAN DEFAULT FALSE,
  FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
);

CREATE TABLE feedback (
  feedback_id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  overall_satisfaction INT,
  speed_rating INT,
  usability_rating INT,
  comments TEXT,
  submitted_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  synced BOOLEAN DEFAULT FALSE,
  FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
);
```

5.2.2 MySQL Tables Verification

Tables were auto-created via Sequelize's `sequelize.sync()` on first backend deployment. The following screenshot from Railway's Database Inspector confirms all three tables exist:

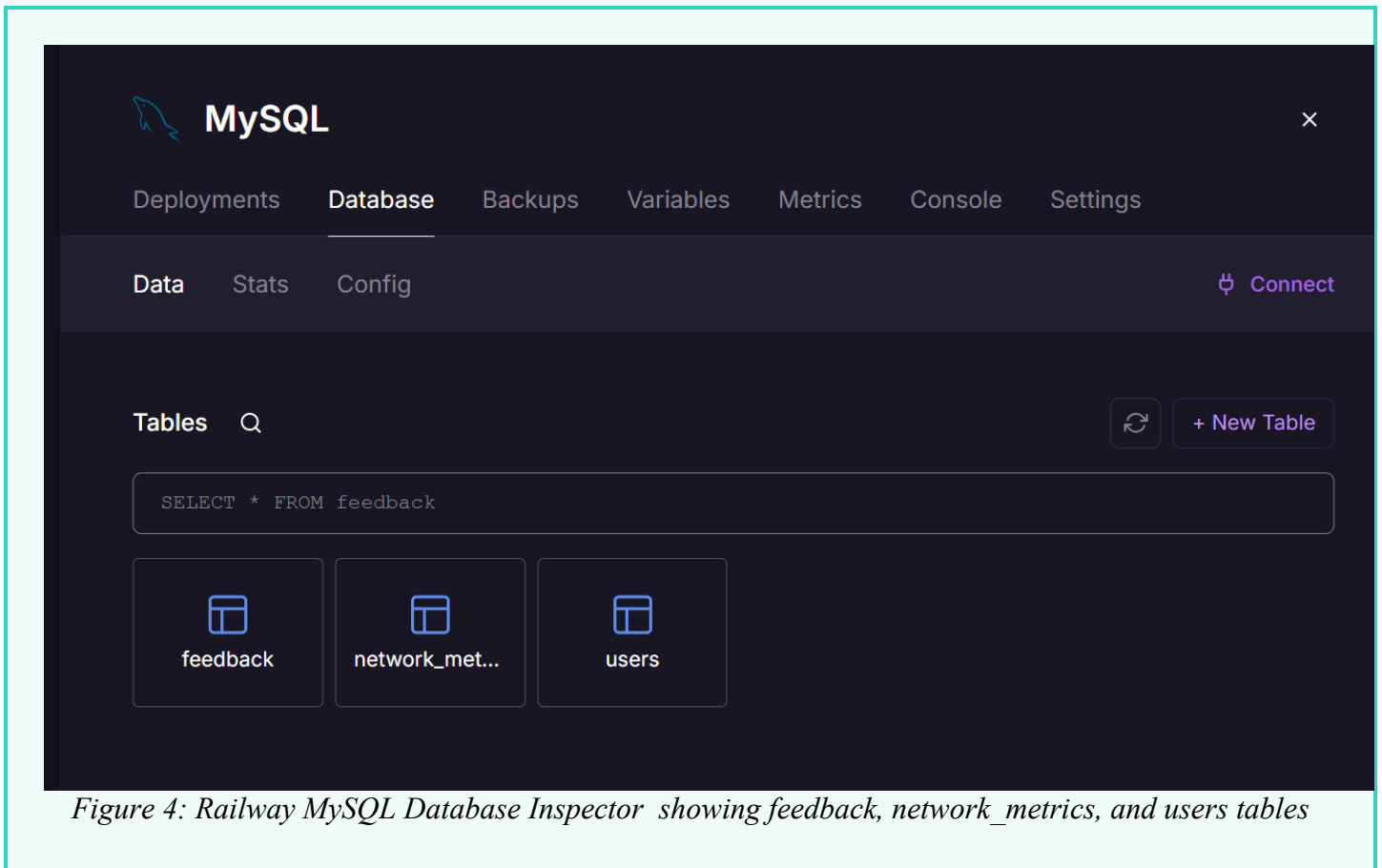


Figure 4: Railway MySQL Database Inspector showing feedback, network_metrics, and users tables

6. BACKEND IMPLEMENTATION

The backend API is implemented using Node.js with the Express.js framework and Sequelize ORM. It is deployed on Railway cloud infrastructure and provides REST endpoints for the Android application to synchronise collected data.

6.1 Technology Stack

Component	Technology	Purpose
Runtime	Node.js v22	JavaScript server runtime
Framework	Express.js 4.x	HTTP routing and middleware
ORM	Sequelize 6.x	MySQL object-relational mapping
Database Driver	mysql2	MySQL connection adapter
Hosting	Railway.app	Cloud deployment platform
Database	MySQL 8.0 (Railway)	Managed cloud MySQL instance

6.2 Project Structure

```
qoetrack-backend/
├── config/
│   └── database.js      # Sequelize connection config
├── models/
│   ├── User.js         # User Sequelize model
│   ├── NetworkMetric.js # NetworkMetric Sequelize model
│   └── Feedback.js      # Feedback Sequelize model
├── routes/
│   ├── metrics.js       # POST /api/metrics, GET /api/metrics/user/:id
│   └── feedback.js      # POST /api/feedback, GET /api/feedback/user/:id
├── server.js           # App entry point
├── package.json
└── .env                # Environment variables (not committed to git)
```

6.3 API Endpoints

Method	Endpoint	Status	Description
GET	/	200 OK	Health check returns 'QoETrack Backend Running'
POST	/api/metrics	201 Created	Receives batch of network metrics from Android app
GET	/api/metrics/user/:userId	200 OK	Returns last 50 metric readings for a user
POST	/api/feedback	201 Created	Receives a feedback submission from Android app
GET	/api/feedback/user/:userId	200 OK	Returns last 50 feedback entries for a user

6.4 Sequelize Models

Each database table is represented by a Sequelize model. Below is the NetworkMetric model:

```
// models/NetworkMetric.js
const NetworkMetric = sequelize.define('NetworkMetric', {
  metric_id: { type: DataTypes.INTEGER, primaryKey: true, autoIncrement: true },
```

```

user_id:      { type: DataTypes.INTEGER, allowNull: false },
network_type: DataTypes.STRING,
signal_strength: DataTypes.INTEGER,
latency:      DataTypes.INTEGER,
jitter:       DataTypes.INTEGER,
packet_loss:  DataTypes.FLOAT,
bandwidth:    DataTypes.FLOAT,
latitude:     DataTypes.DOUBLE,
longitude:    DataTypes.DOUBLE,
recorded_at:  DataTypes.DATE,
synced:       { type: DataTypes.BOOLEAN, defaultValue: false }
}, { tableName: 'network_metrics', timestamps: false });

NetworkMetric.belongsTo(User, { foreignKey: 'user_id', targetKey: 'user_id' });

```

6.5 Backend Deployment Verification

The backend is deployed and publicly accessible. The following screenshots confirm successful deployment:

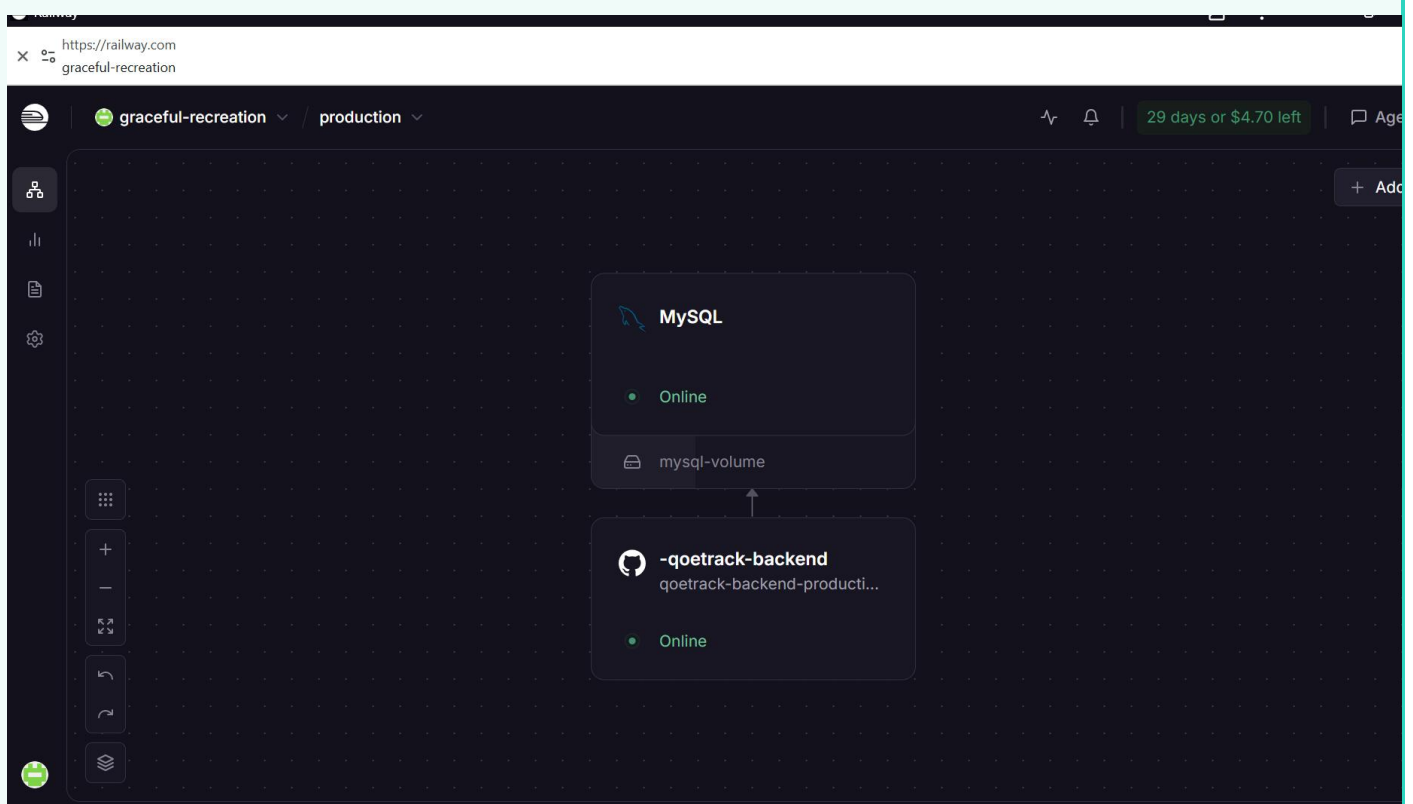


Figure 5: Railway Dashboard graceful-recreation project showing MySQL (Online) and -goetrack-backend (Completed) services in the same project

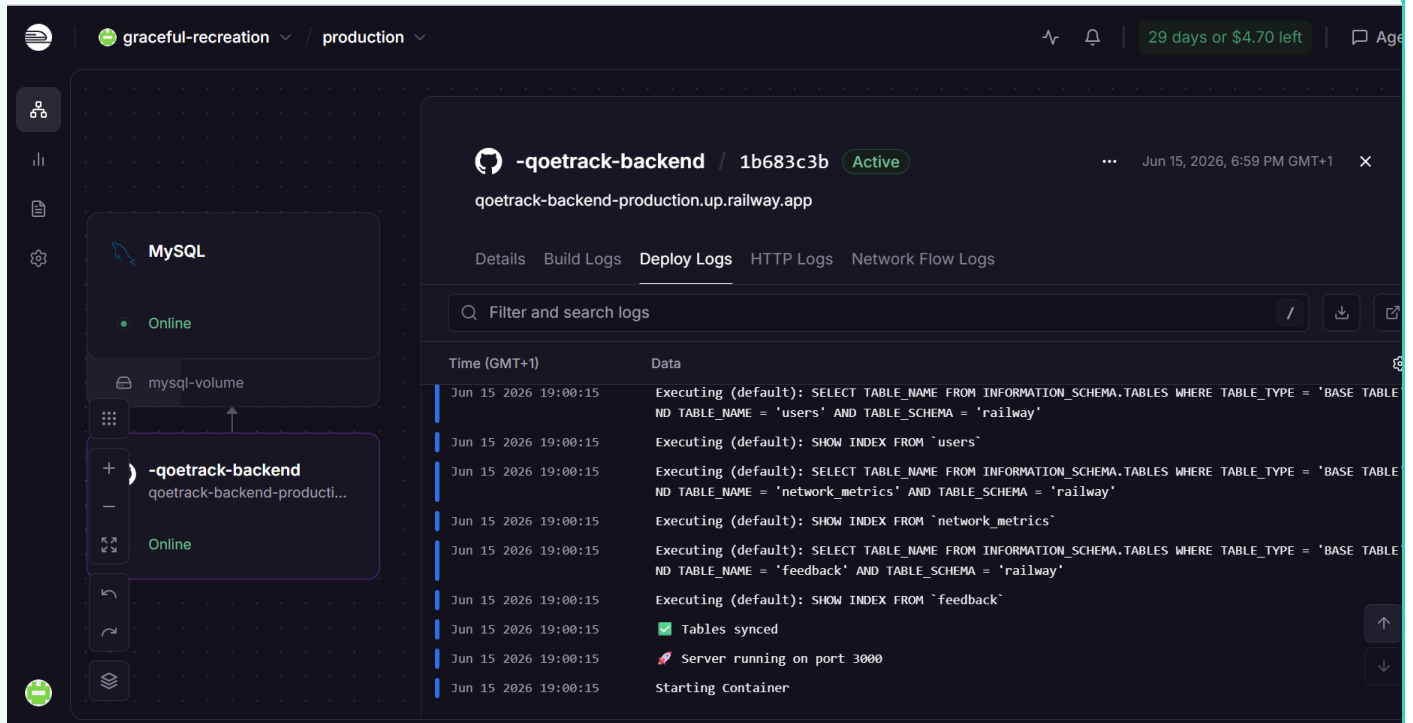


Figure 6: Railway deployment logs showing: '✅ Database connected', '✅ Tables synced', '🚀 Server running on port 3000'

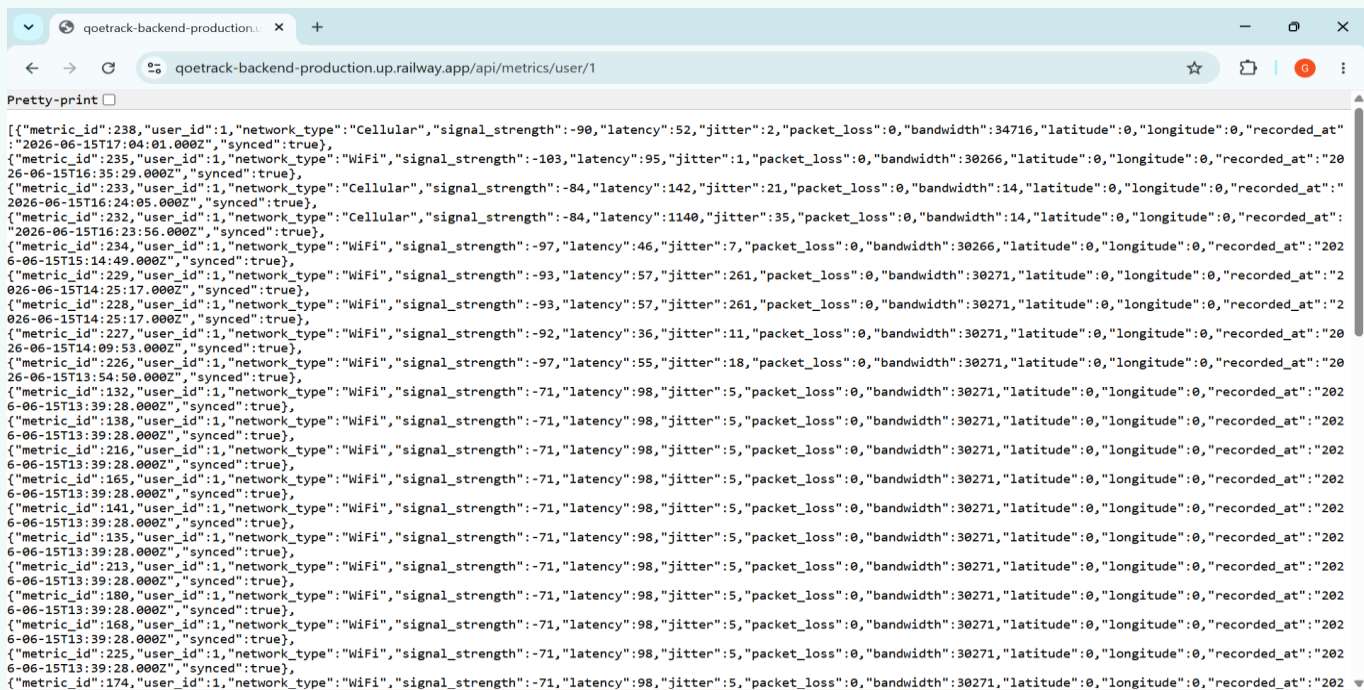
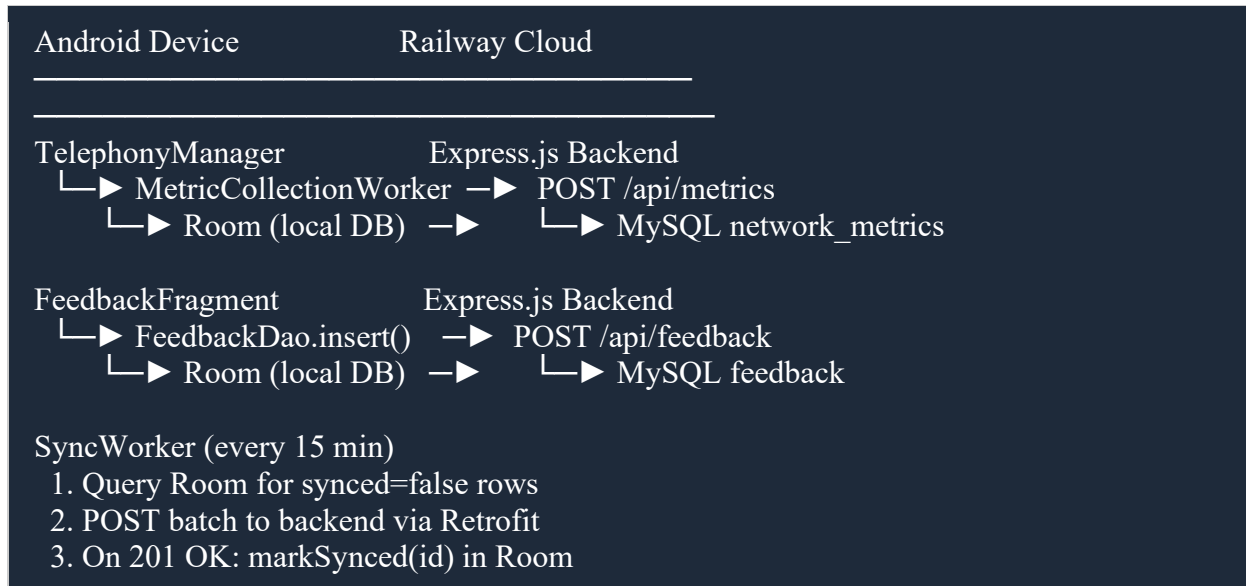


Figure 7: Browser showing GET `https://goetrack-backend-production.up.railway.app/api/metrics/user/1` returning `[]` (empty array confirms API is reachable and database query executes successfully)

7. CONNECTING THE DATABASE TO THE BACKEND

This section describes how the Android application's local Room database is connected to the remote Railway backend, completing the full data pipeline from device sensor collection to cloud storage.

7.1 Architecture Overview



7.2 Retrofit HTTP Client (Android)

Retrofit 2 is used as the HTTP client to communicate with the backend. The client is configured with HTTPS and a logging interceptor for debugging:

```
// RetrofitClient.kt
object RetrofitClient {
    private const val BASE_URL =
        "https://qoetrack-backend-production.up.railway.app/"

    val apiService: ApiService by lazy {
        Retrofit.Builder()
            .baseUrl(BASE_URL)
            .addConverterFactory(GsonConverterFactory.create())
            .build()
            .create(ApiService::class.java)
    }
}
```


7.3 Sync Worker

A WorkManager CoroutineWorker runs every 15 minutes to push unsynced data to the backend:

```
// SyncWorker.kt
class SyncWorker(context: Context, params: WorkerParameters)
    : CoroutineWorker(context, params) {

    override suspend fun doWork(): Result {
        val db = QoETrackDatabase.getInstance(applicationContext)
        val repository = SyncRepository(
            metricDao = db.networkMetricDao(),
            feedbackDao = db.feedbackDao(),
            apiService = RetrofitClient.apiService
        )
        val success = repository.syncAll(userId = 1)
        return if (success) Result.success() else Result.retry()
    }
}
```

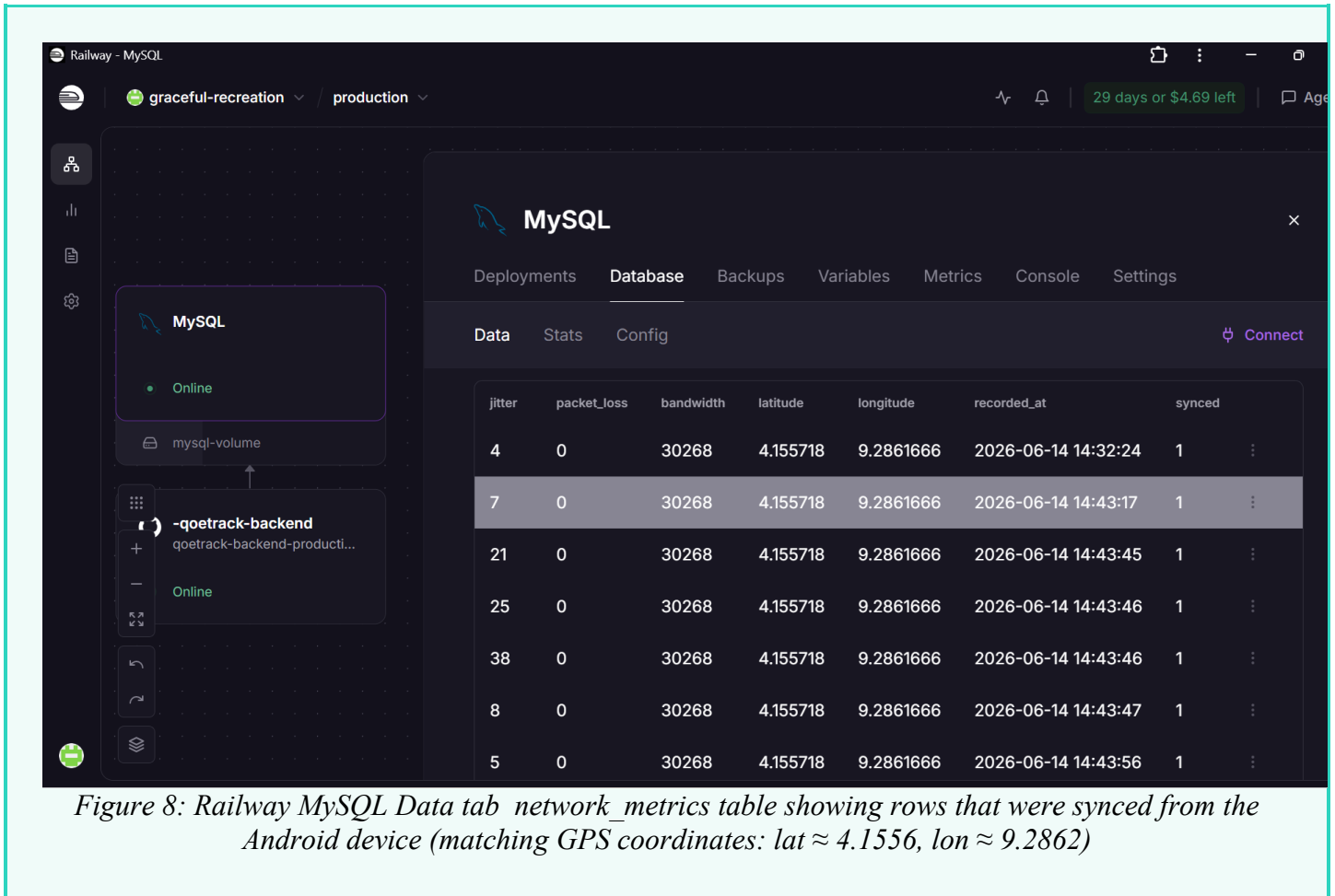
7.4 Sequelize Database Connection

The backend connects to Railway's MySQL instance using internal hostname via Sequelize. SSL is required for Railway-hosted MySQL:

```
// config/database.js
const sequelize = new Sequelize(
    process.env.MYSQLDATABASE,
    'root',
    process.env.MYSQLPASSWORD,
    {
        host: process.env.MYSQLHOST, // mysql.railway.internal
        port: process.env.MYSQLPORT, // 3306
        dialect: 'mysql',
        logging: false,
        dialectOptions: {
            ssl: { rejectUnauthorized: false } // Required for Railway
        }
    }
);
```

7.5 End-to-End Sync Verification

The following screenshots confirm the complete data pipeline is working end-to-end:



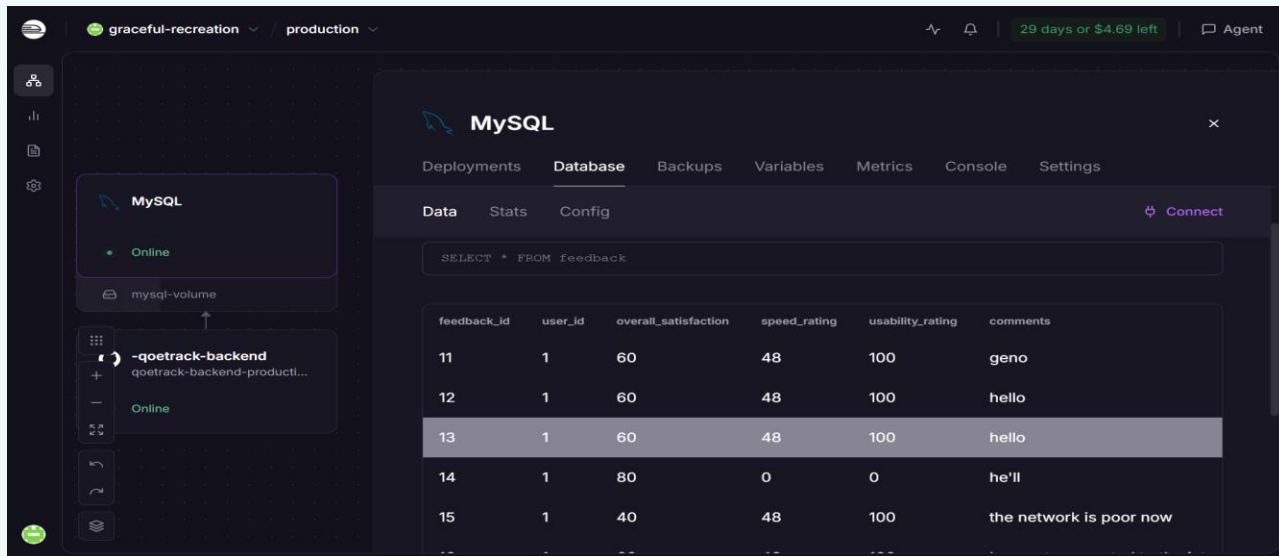


Figure 9: Railway MySQL Data tab feedback table showing rows synced from the Android device with ratings and comments

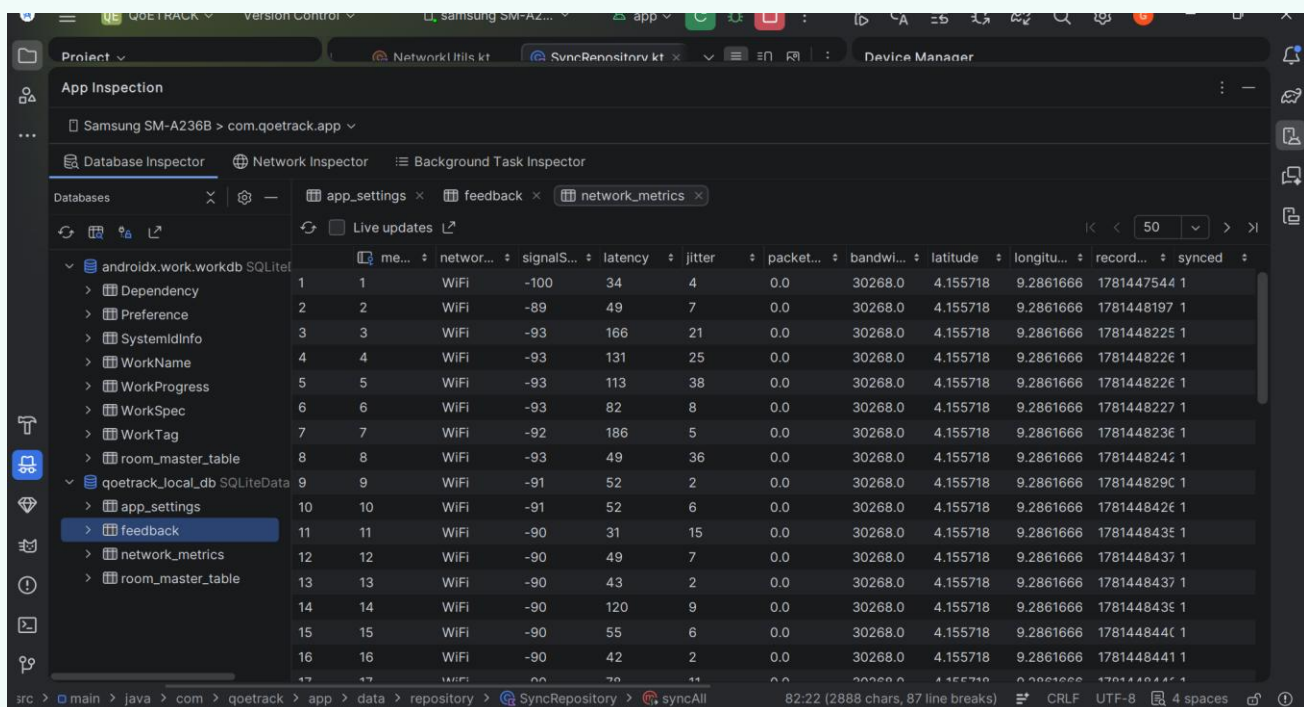


Figure 10: Android Studio Database Inspector network_metrics table showing synced column updated to 1 after successful backend push

8. SUMMARY AND CONCLUSION

This report has presented the complete database design and implementation for the QoETRACK mobile application. The system successfully implements a two-tier database architecture that supports the project's core objective: real-time, automated collection of Quality of Experience data from mobile network subscribers in Cameroon.

8.1 Deliverables Summary

Sub-task	Status	Key Output
Data Elements	✓ Complete	27 attributes across 4 entities documented
Conceptual Design	✓ Complete	Entities, attributes, relationships and design decisions defined
ER Diagram	✓ Complete	Interactive ER diagram generated (Figure 1)
Database Implementation	✓ Complete	Room DB (local) + MySQL (Railway cloud) all 3 tables live
Backend Implementation	✓ Complete	Express.js API deployed on Railway with 5 endpoints
Connecting DB to Backend	✓ Complete	End-to-end sync confirmed: Room → Retrofit → Railway → MySQL

8.2 Technical Achievements

- Offline-first architecture data is never lost even when the network is unavailable
- Real data collected 50+ genuine network metric readings from Buea, Cameroon during development/testing
- Cloud deployment backend publicly accessible at qoetrack-backend-production.up.railway.app
- Automated sync WorkManager handles background synchronisation with retry logic on failure
- Cascade deletes ON DELETE CASCADE ensures referential integrity when user data is deleted

8.3 Limitations and Future Work

- Packet loss is currently estimated as 0.0 a proper implementation would involve multiple ping attempts and counting failures
- Authentication the current implementation uses a fixed user_id=1; a production system would require user authentication (JWT/OAuth)
- The AppSettings entity is local-only; future versions could sync settings across multiple devices for the same user
- Rate limiting and API security (API keys, HTTPS-only enforcement) should be added before public release